TRANSACCIONES.

operaciones sobre tablas que deban ir juntas, o se ejecutan todas o ninguna.
Por ejemplo:

a) Hago una venta de algo y actualiza las existencias. (insert en venta y update en artículos)
b) Hago una compra de un trayecto en dos tramos, por ejemplo viaje de Málaga a NYC vía londres: AGP -> LON → NY (2 insert en la tabla de los boletos)

Tabla ventas: id*, id_articulo, cantidad, fecha.
Tabla articulos: Id*, nombre, existencias, precio.

para tratar transacciones, debemos:

- desactivar autocommit,
- Marcar el inicio de la transacción
- Si todo va bien al modificar las tablas, damos commit
- Si hay algún percance, deshacemos cambios y lo dejamos en el estado previo (rollback)

Código

1. **Configuración de la Conexión**:
   - Se usa `DriverManager.getConnection` para conectar con la base de datos MySQL.
   - No cambia con respecto a otros ejercicios JDBC
2. **Desactivar AutoCommit**:
   - `conexion.setAutoCommit(false)` permite manejar la transacción manualmente. Por defecto está activo.
3. **Operaciones de la Transacción**:
   - **Insertar en** `ventas`: Se registra una venta con el ID del artículo, la cantidad vendida .
   - **Actualizar en** `articulos`: Se resta la cantidad vendida de las existencias del artículo correspondiente.
4. **Commit o Rollback**:
   - Si ambas operaciones son exitosas, se ejecuta `conexion.commit()` para confirmar.
   - Si ocurre una circunstancia o error (por ejemplo, si no hay suficientes existencias), se ejecuta `conexion.rollback()` para deshacer los cambios.
5. **Cierre de Recursos**:
   - En el bloque `finally`, se restaura el `autoCommit` y se cierra la conexión.

Código de ejemplo de venta y actualización

_____JAVA_____

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class TransaccionVentas {
    // Configuración de la conexión a MySQL
    private static final String URL = "jdbc:mysql://localhost:3306/tu_base_de_datos";
    private static final String USER = "tu_usuario";
    private static final String PASSWORD = "tu_contraseña";

    public static void main(String[] args) {
        Connection conexion = null;

        try {
            // Establecer conexión
            conexion = DriverManager.getConnection(URL, USER, PASSWORD);
            // Desactivar autocommit para manejar transacciones manualmente
            conexion.setAutoCommit(false);

            // sql 1: Insertar una nueva venta
            String sql1 = "INSERT INTO ventas (articulo_id, cantidad, fecha) VALUES (?, ?, NOW())";
            PreparedStatement stmt1 = conexion.prepareStatement(sql1);
            stmt1.setInt(1, 1); // ID del artículo
            stmt1.setInt(2, 5); // Cantidad vendida
            stmt1.executeUpdate();


            // sql 2: Actualizar existencias en la tabla de artículos
            String sql2 = "UPDATE articulos SET existencias = existencias - ? WHERE id = ?";
            PreparedStatement stmt2 = conexion.prepareStatement(sql2);
            stmt2.setInt(1, 5); // Cantidad a restar
            stmt2.setInt(2, 1); // ID del artículo
            stmt2.executeUpdate();

            // Confirmar la transacción
            conexion.commit();
            System.out.println("Transacción completada: Venta registrada y existencias actualizadas.");

        } catch (SQLException e) {
            // En caso de error, revertir la transacción
            try {
                if (conexion != null) {
                    conexion.rollback();
                    System.out.println("Transacción revertida debido a un error.");
                }
            } catch (SQLException rollbackEx) {
```

```
                rollbackEx.printStackTrace();
            }
            e.printStackTrace();
        } finally {
            // Restaurar autocommit y cerrar conexión
            try {
                if (conexion != null) {
                    conexion.setAutoCommit(true);
                    conexion.close();
                }
            } catch (SQLException closeEx) {
                closeEx.printStackTrace();
            }
        }
    }
}
```

PROCEDIMIENTOS ALMACENADOS

Supongamos una BD de agenda con una tabla donde insertamos mediante un procedimiento:

tabla en mysql

```
CREATE TABLE agenda_telefonica (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50),
    telefono VARCHAR(15)
);
```

procedimiento simple para insertar una tabla

```
DELIMITER //

CREATE PROCEDURE insertarContacto(
    IN p_nombre VARCHAR(50),
    IN p_telefono VARCHAR(15)
)
BEGIN
    INSERT INTO agenda_telefonica (nombre, telefono)
    VALUES (p_nombre, p_telefono);
END //

DELIMITER ;
```

Ejecutar procedimiento desde java

Es similar a una operacion normal de sql, pero usando objetos CallableStatement en lugar de PreparedStatement.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.CallableStatement;
import java.sql.SQLException;

public class AgendaTelefonica {

    // Configuración de la base de datos
    private static final String URL = "jdbc:mysql://localhost:3306/tu_base_de_datos";
    private static final String USER = "usuario";
```

```
    private static final String PASSWORD = "contraseña";

    public static void main(String[] args) {
        // Datos del nuevo contacto
        String nombre = "Pepe Perez";
        String telefono = "123456789";

        // Llamada a la funcion que ejecuta el  procedimiento para insertar el contacto
        insertarContacto(nombre, telefono);
    }

    public static void insertarContacto(String nombre, String telefono) {
        Connection conn = null;
        CallableStatement stmt = null;

        try {
            // Conectar a la base de datos
            conn = DriverManager.getConnection(URL, USER, PASSWORD);

            // Preparar la llamada al procedimiento almacenado
            String sql = "{CALL insertarContacto(?, ?)}";
            stmt = conn.prepareCall(sql);

            // Establecer los parámetros de entrada
            stmt.setString(1, nombre);
            stmt.setString(2, telefono);

            // Ejecutar el procedimiento almacenado
            stmt.execute();

            System.out.println("Contacto insertado con éxito.");

        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                // Cerrar el statement y la conexión
                if (stmt != null) stmt.close();
                if (conn != null) conn.close();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

Procedimiento para insertar con encriptacion de clave

```
DELIMITER //

CREATE PROCEDURE insertar_en_agenda(
    IN p_nombre VARCHAR(100),
    IN p_contraseña VARCHAR(100),
    IN p_telefono VARCHAR(15)
)
BEGIN
    INSERT INTO agenda (nombre, contraseña, telefono)
    VALUES (p_nombre, MD5(p_contraseña), p_telefono);
END //

DELIMITER ;
```

Y llamada desde java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.CallableStatement;
import java.sql.SQLException;

public class AgendaApp {
    // Datos de conexión a la base de datos
    private static final String URL = "jdbc:mysql://localhost:3306/tu_base_de_datos";
    private static final String USER = "tu_usuario";
    private static final String PASSWORD = "tu_contraseña";

    public static void main(String[] args) {
        String nombre = "Juan Pérez";
        String contraseña = "mi_secreta_contraseña";
        String telefono = "555-1234";

        insertarEnAgenda(nombre, contraseña, telefono);
    }

    public static void insertarEnAgenda(String nombre, String contraseña, String telefono) {
        // Conexión a la base de datos
        try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD)) {
            // Llama al procedimiento almacenado
            String sql = "{CALL insertar_en_agenda(?, ?, ?)}";
            try (CallableStatement stmt = conn.prepareCall(sql)) {
                // Asigna los parámetros
                stmt.setString(1, nombre);
```

```
            stmt.setString(2, contraseña);  // La contraseña será encriptada por el
procedimiento en MySQL
            stmt.setString(3, telefono);

            // Ejecuta el procedimiento almacenado
            stmt.execute();
            System.out.println("Registro insertado correctamente.");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
  }
}
```